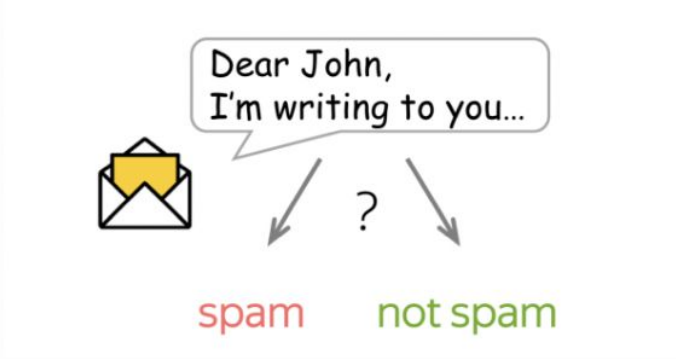
Transfer Learning

Nikolay Karpachev
17.03.2026

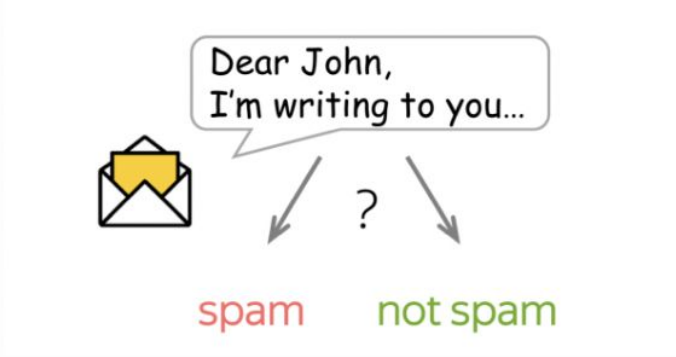
Transfer Learning

Let’s say we need to train a text classification model



Transfer Learning

Let’s say we need to train a text classification model

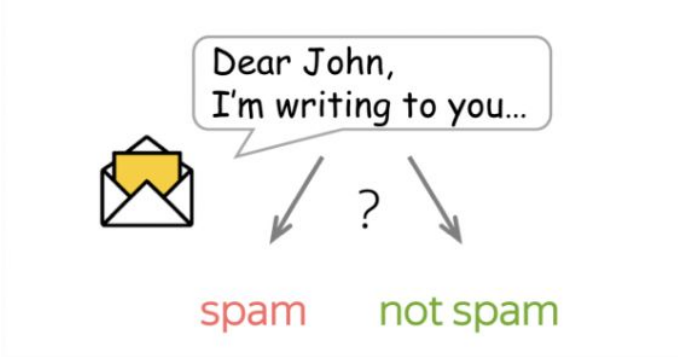


To train from scratch, we need

- Labeled samples
- Pre-extracted features

Transfer Learning

Let’s say we need to train a text classification model



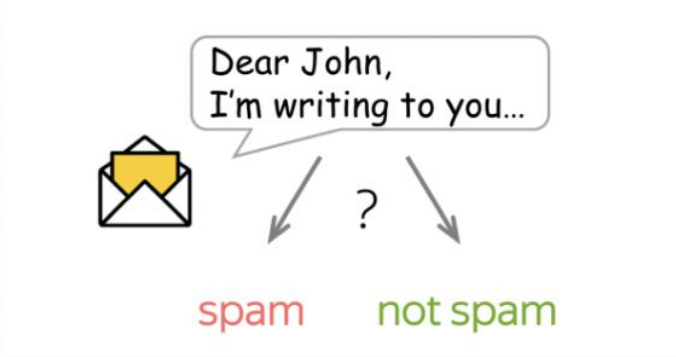
To train from scratch, we need

- Labeled samples
- Pre-extracted features

Expensive to obtain

Transfer Learning

Let's say we need to train a text classification model



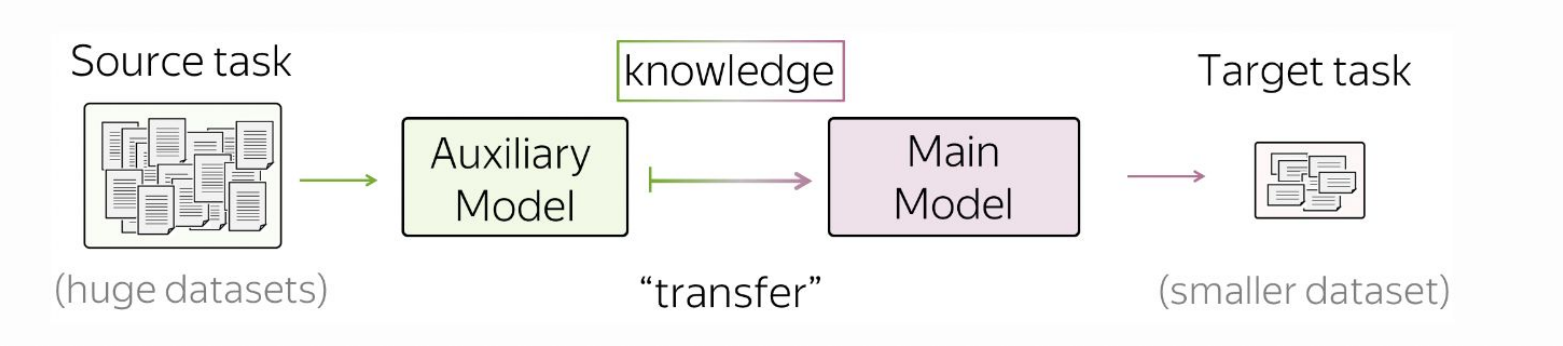
To train from scratch, we need

- Labeled samples
- Pre-extracted features

Expensive to obtain

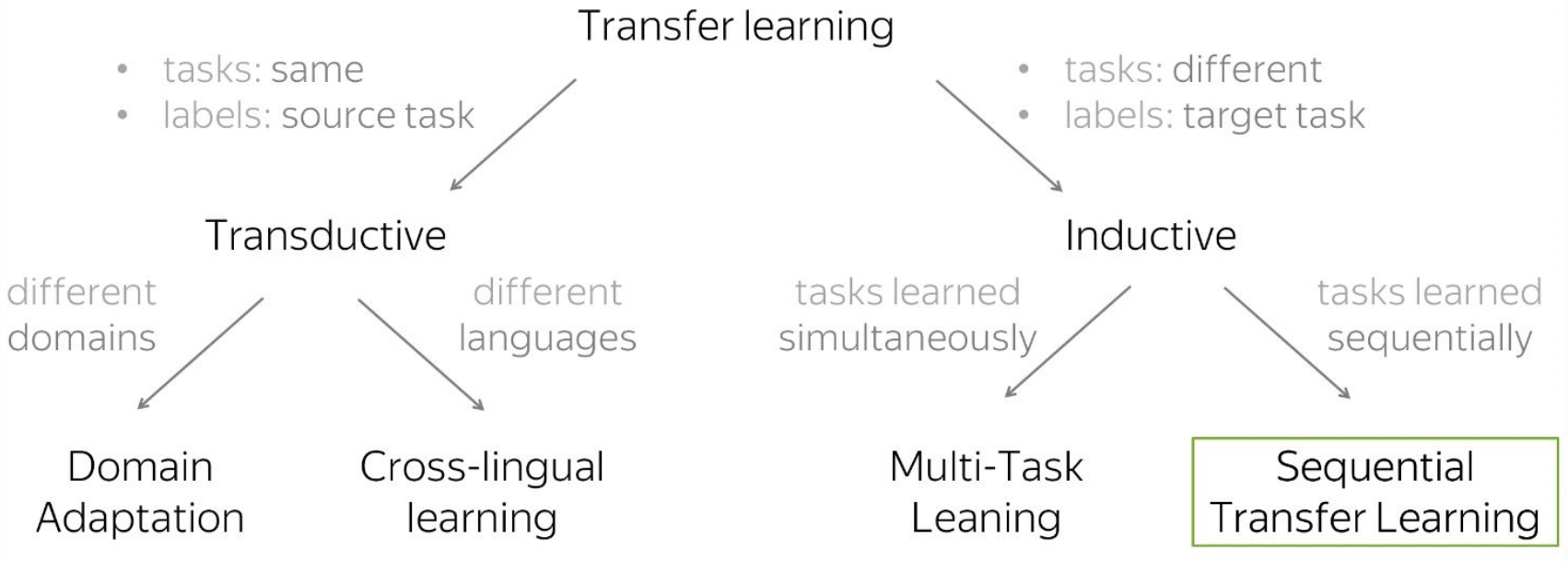
Which features are useful???

Transfer Learning

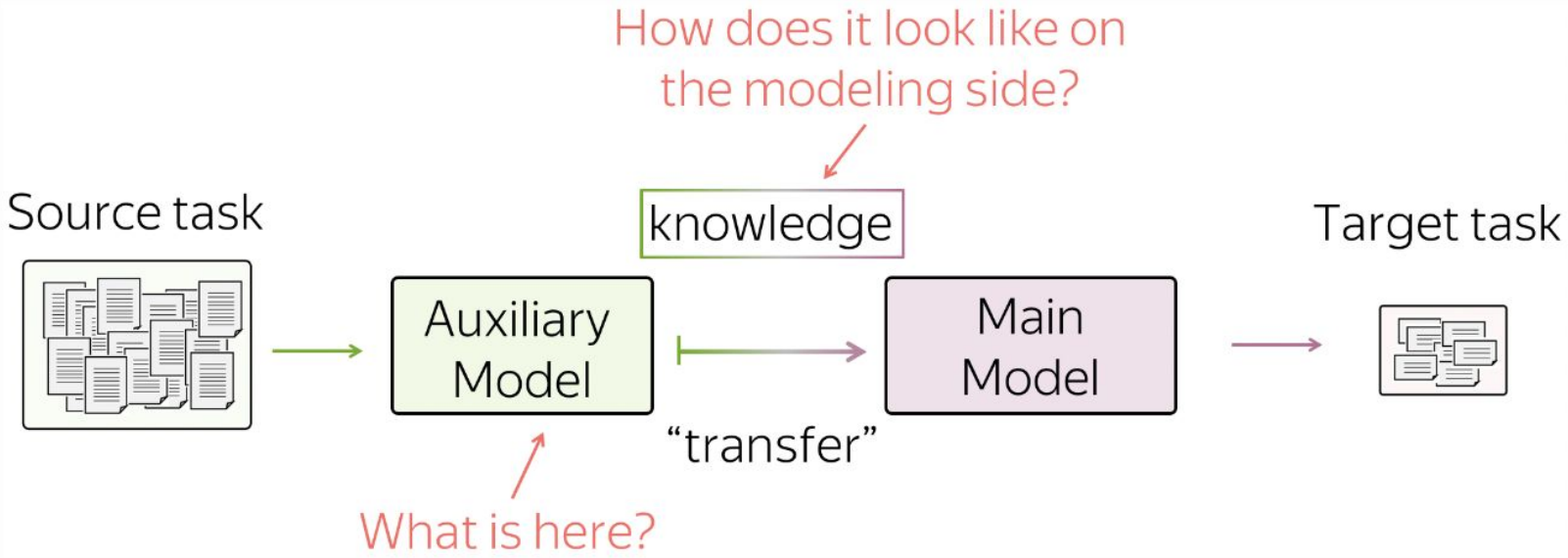


Transfer Learning is a technique of exploiting the properties and data distribution of a given (task, dataset) pair on different tasks and datasets of interest

Transfer Learning Taxonomy

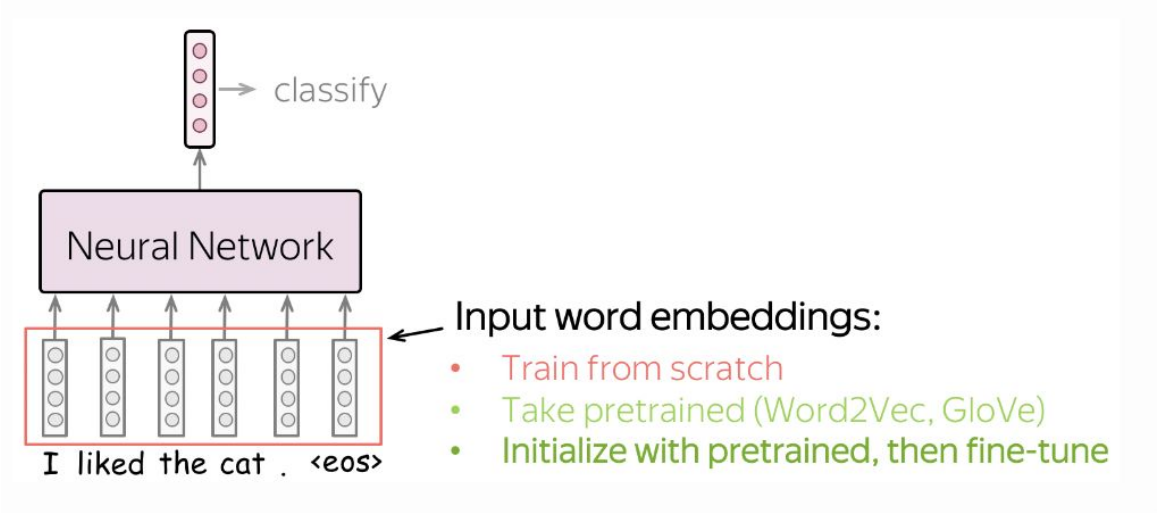


Sequential Transfer Learning



Transfer Learning via Word Embeddings

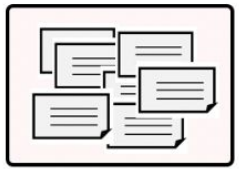
- Let’s look at embedding layer in classification pipeline



Transfer Learning via Word Embeddings

- Which data distribution was embedding layer attuned to?

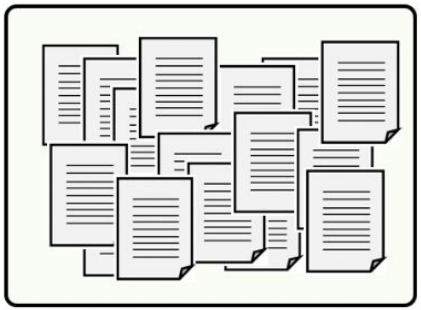
Embedding layer from scratch
(trained jointly for cls)



Training data for text classification (labeled)

- Not huge, or not diverse, or both
- Domain: task-specific

Embedding layer == word2vec



Training data for word embeddings (unlabeled)

- Huge diverse corpus (e.g., Wikipedia)
- Domain: general

Transfer Learning via Word Embeddings

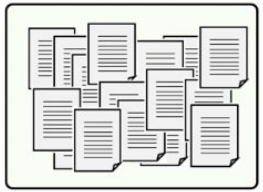
- Train from scratch

What they will know:

May be not enough to learn relationships between words

- Take pretrained (Word2Vec, GloVe)

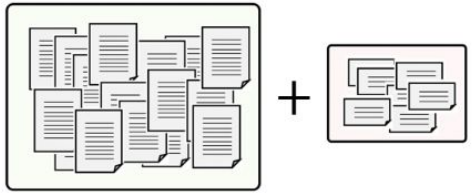
What they will know:



Know relationships between words, but are **not** specific to the task

- **Initialize with pretrained, then fine-tune**

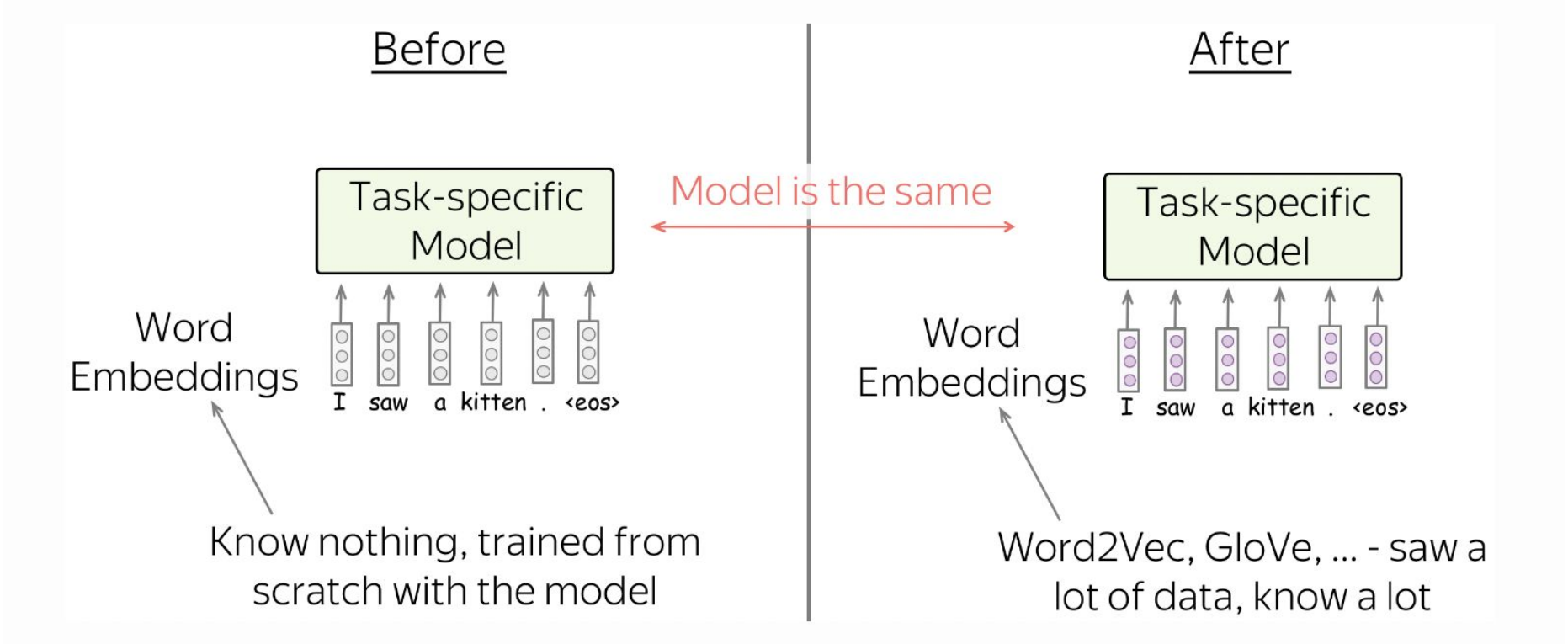
What they will know:



Know relationships between words and adapted for the task

"**Transfer**" knowledge from a huge unlabeled corpus to your task-specific model

Transfer Learning via Word Embeddings

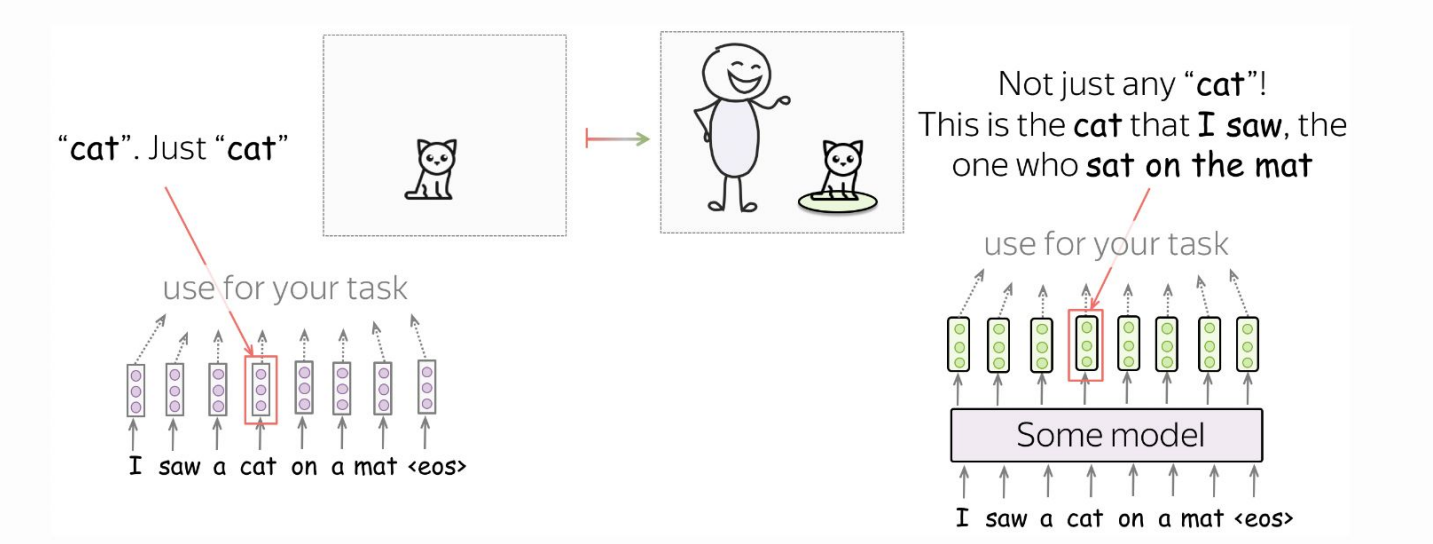


Transfer Learning via Contextualized Representations

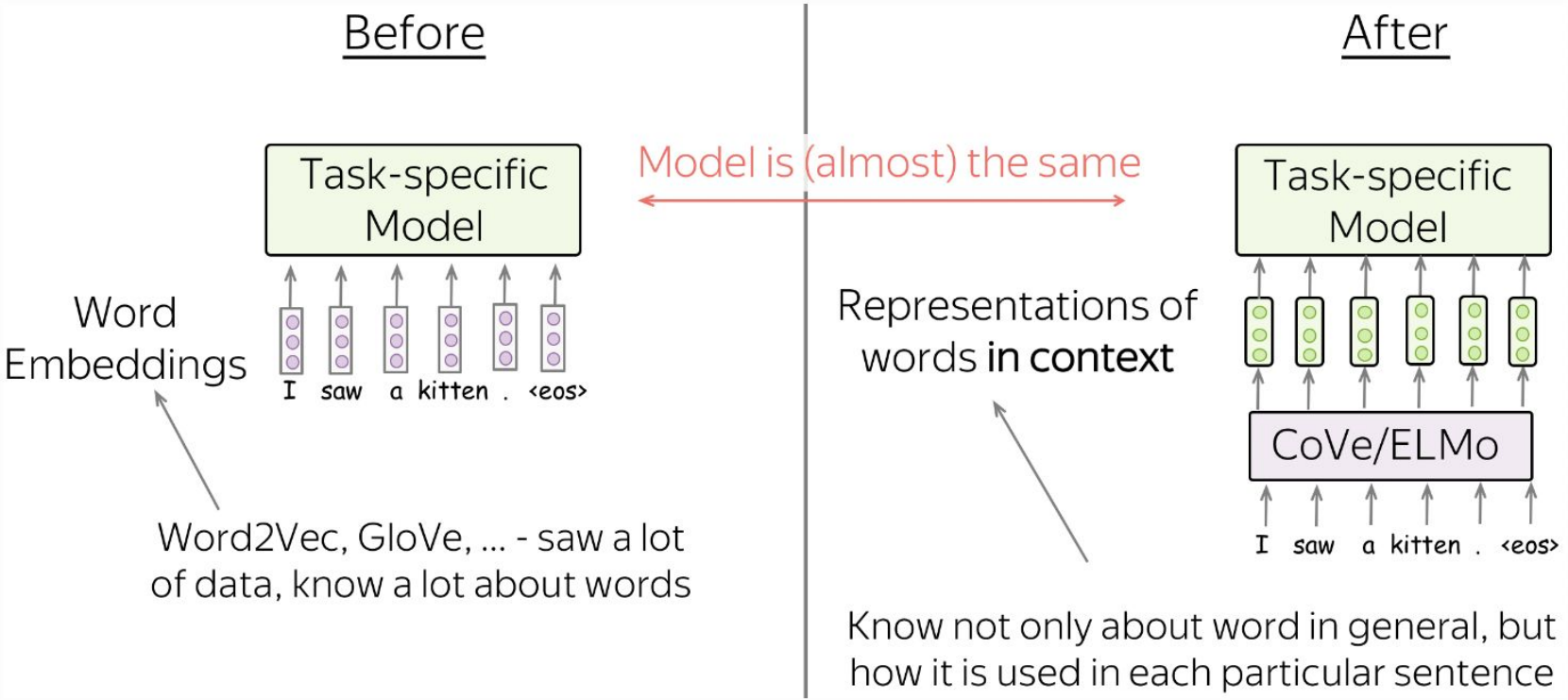
- Word vectors do not model any intra-token relationships

Transfer Learning via Contextualized Representations

- Word vectors do not model any intra-token relationships
- **The same way as words, we can learn to encode words along with the context they are used in**



Transfer Learning via Contextualized Representations



Representation Learning (CoVe, ELMO)

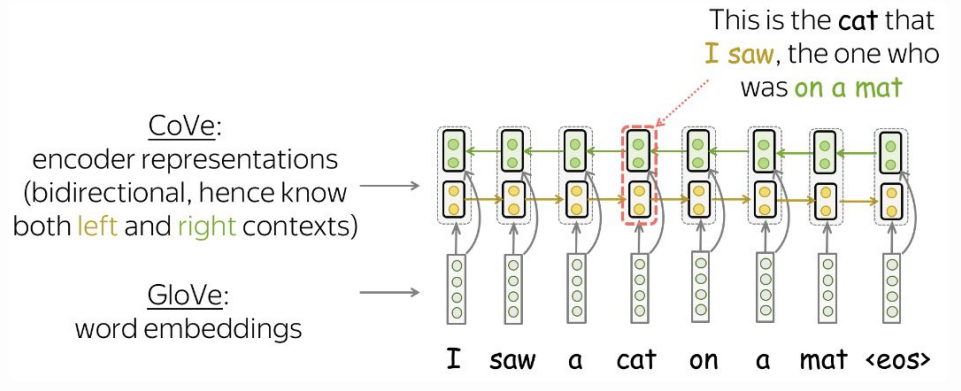
CoVe

CoVe (Contextualized Word Vectors Learned in Translation)

Key Idea: translation of sentence requires modeling complex token dependencies and NMT model learns to “understand” sentence

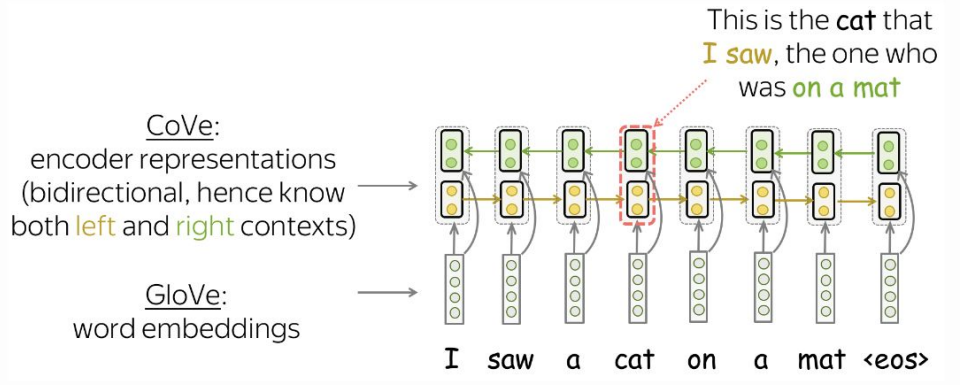
CoVe

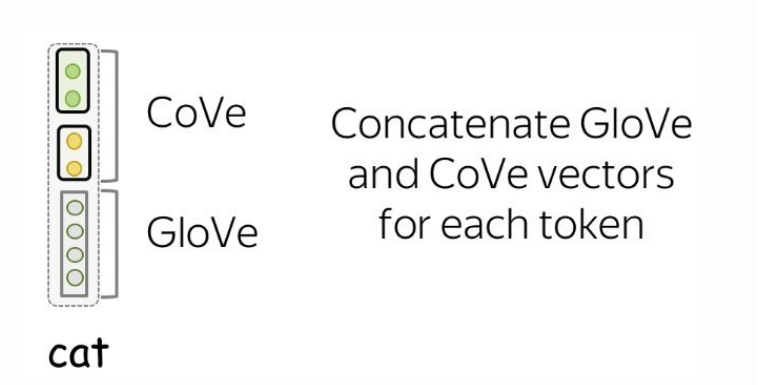
- Train NMT encoder-decoder system
- Take pre-trained encoder as feature extractor
- CoVe = encoder ouputs for a given sequence



CoVe

- Train NMT encoder-decoder system
- Take pre-trained encoder as feature extractor
- CoVe = encoder ouputs for a given sequence
- For downstream tasks: CoVe + GloVe embeddings





ELMO

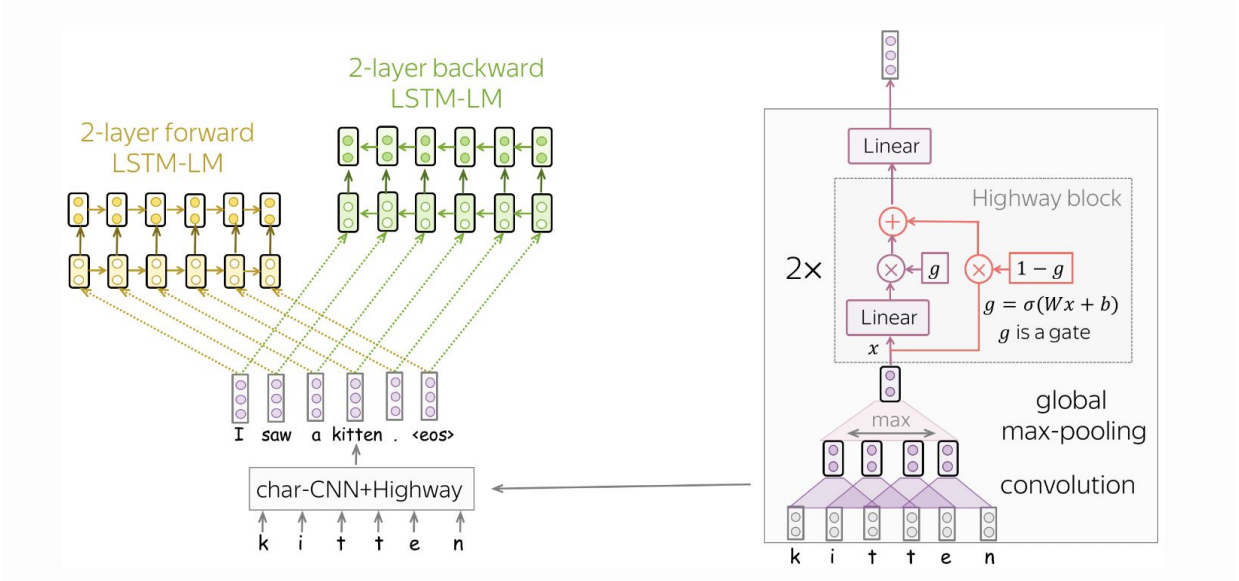
ELMO (Embeddings From Language Models)

Key Idea: similar to CoVe, but instead of NMT use LM pretraining objective

ELMO

ELMO (Embeddings From Language Models)

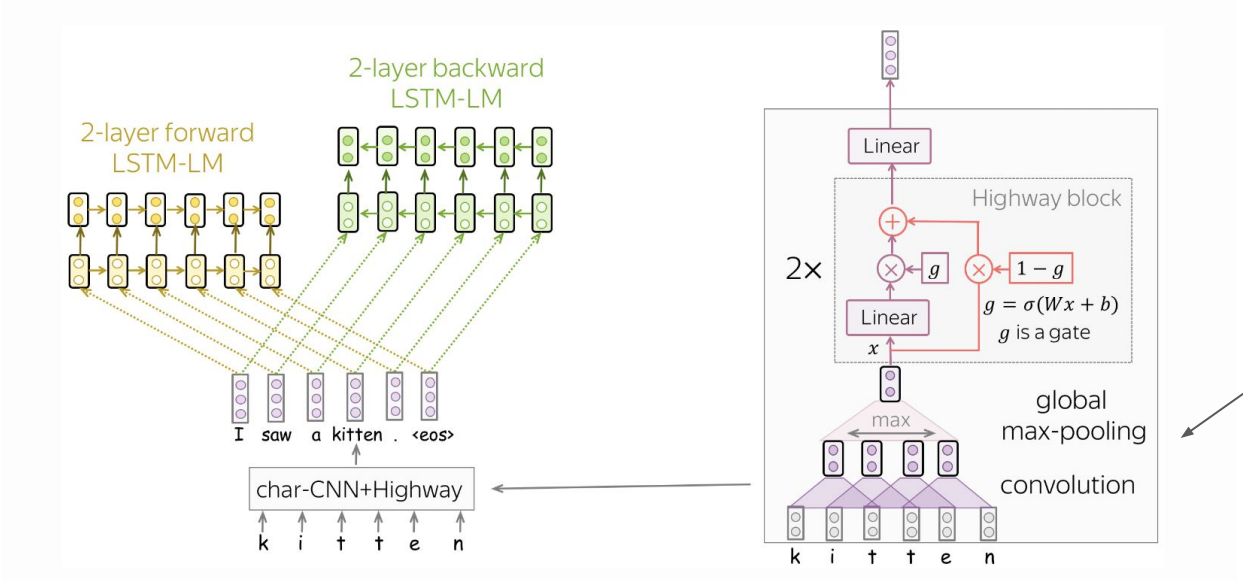
Key Idea: similar to CoVe, but instead of NMT use LM pretraining objective



ELMO

ELMO (Embeddings From Language Models)

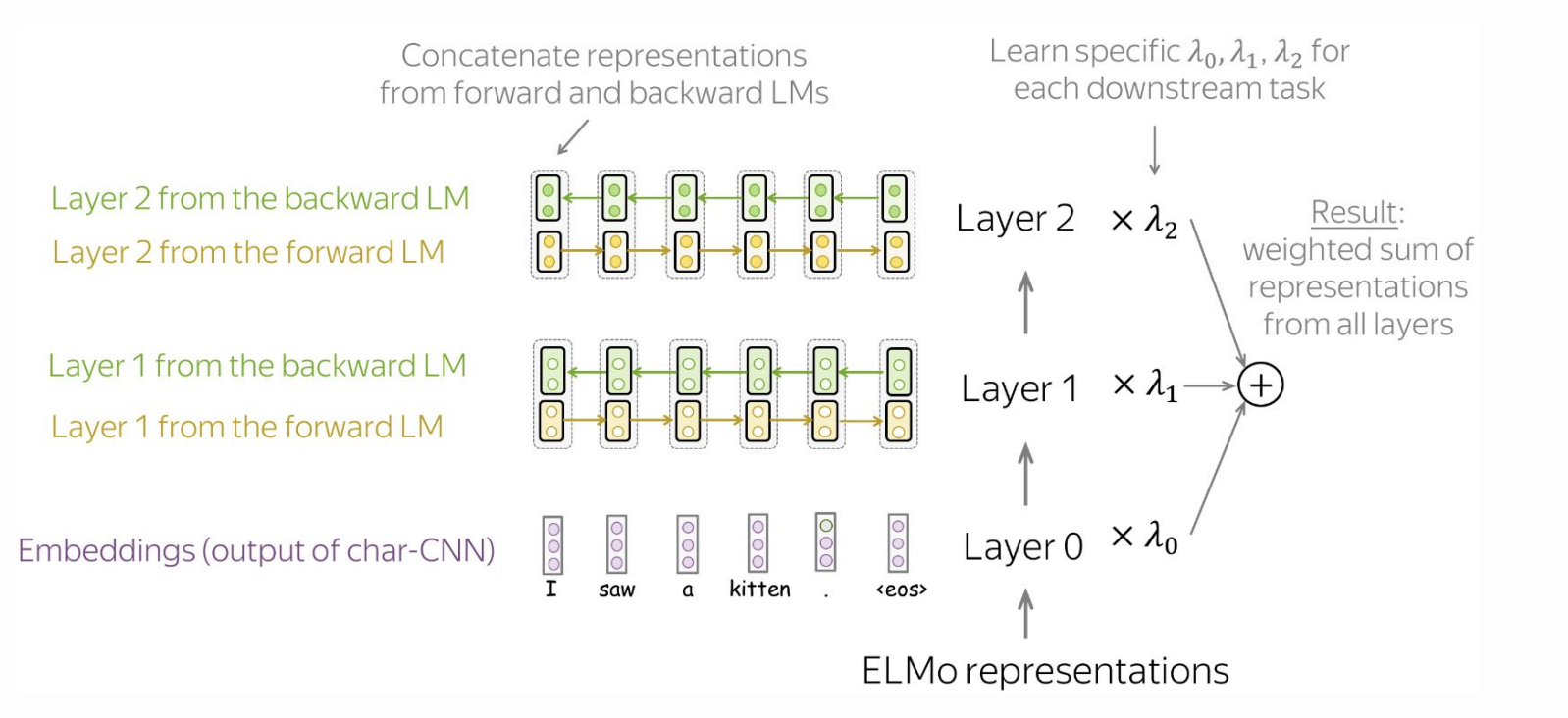
Key Idea: similar to CoVe, but instead of NMT use LM pretraining objective



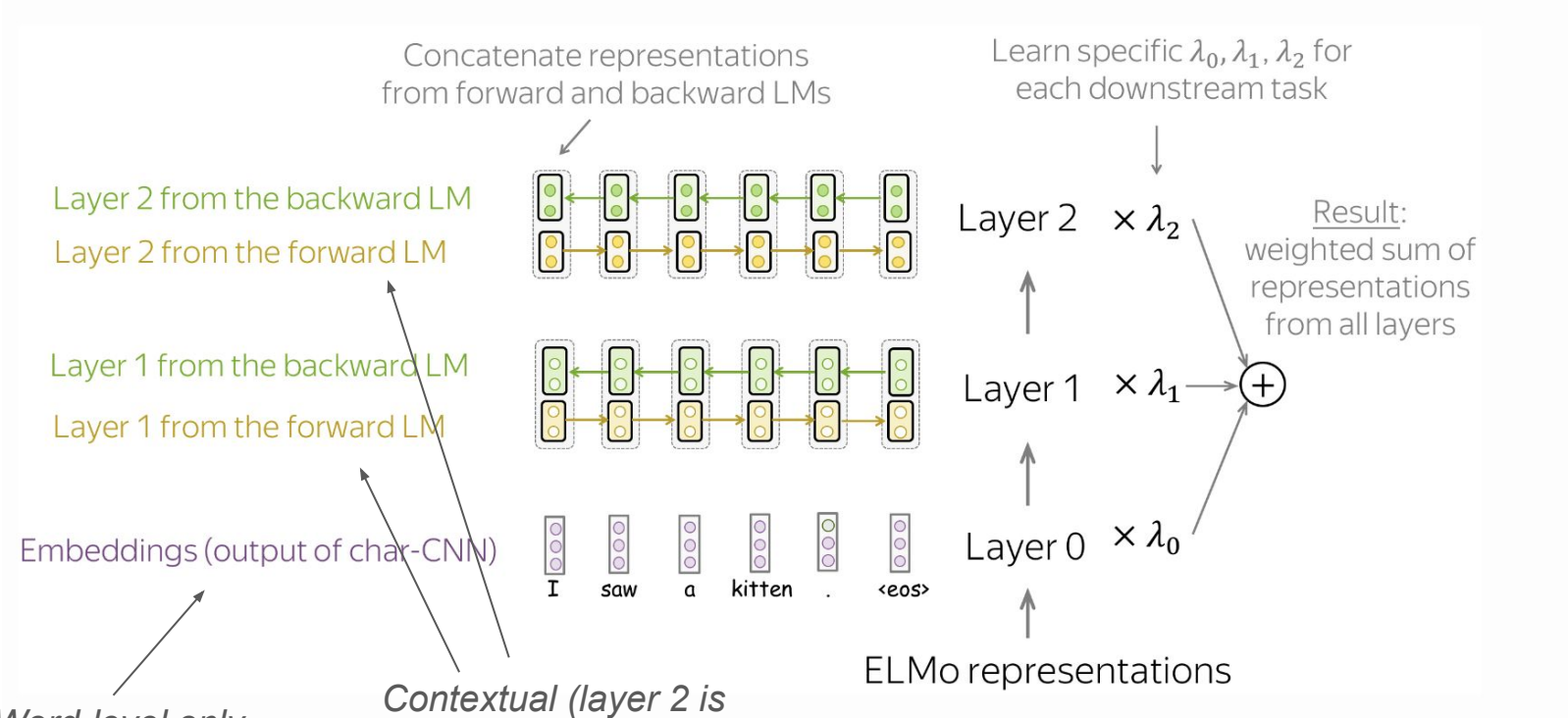
Initialization

Character-level CNN (robust handling of OOV words)

ELMO: Feature Extraction



ELMO: Feature Extraction



Transfer Learning via Pretrained Models

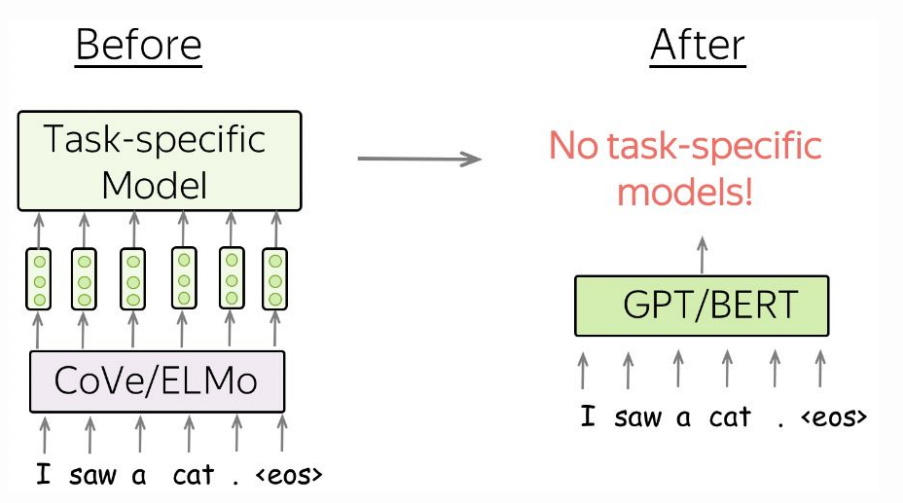
Transfer Learning

Contextualized Representations

- Replace word embedding layer
- Rest of the pipeline should be trained independently for each task

Pretrained Models

- Replace full task-specific model
- Minimal adaptation or usage “as is”



Pretrained Models From Language Modeling

Language Modeling is a fertile task for representation learning

- Semi-supervised (labels are implicitly given in data)
- Ubiquitous datasets (web crawl, literature, etc.)
- Complicated and very generic

GPT

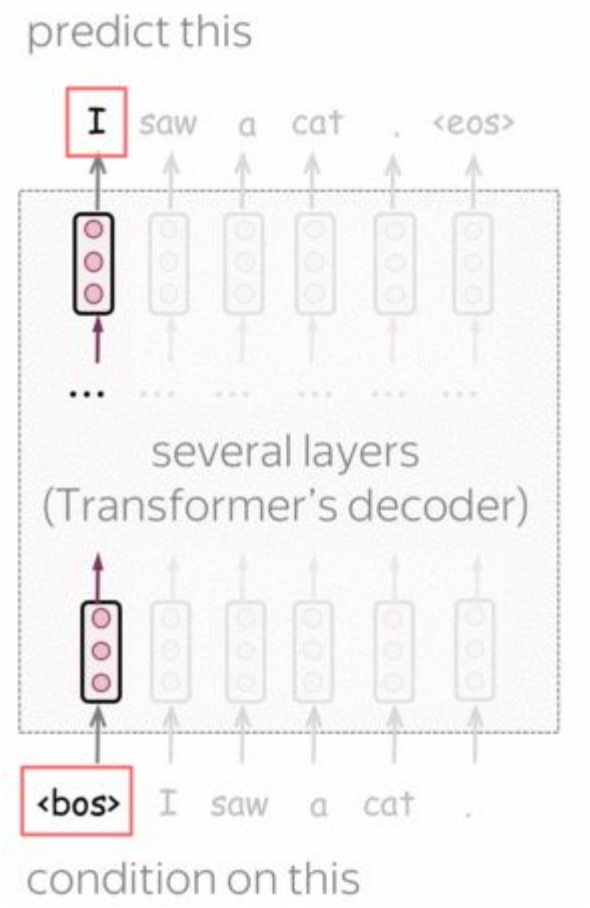
GPT (Generative Pre-Training for Language Understanding)

Key Idea: autoregressive LM with transformer decoder

$$L_{xent} = -\sum_{t=1}^{n} \log(p(y_t|y_{<t})).$$

GPT

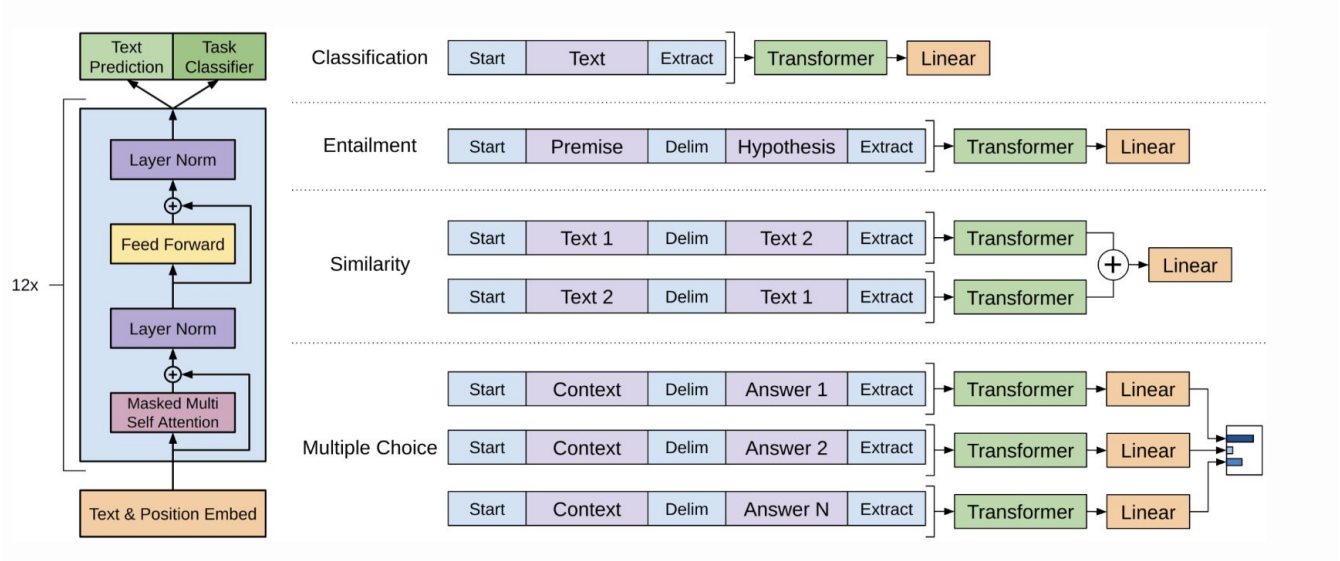
- Decoder-only (triangle attention mask)
- Autoregressively predict next token conditioned on left context
- Train on unsupervised corpora with supervised xent loss (*self-supervised learning*)



GPT Finetuning (GPT-1)

- Finetune LM parameters using a combination of two losses

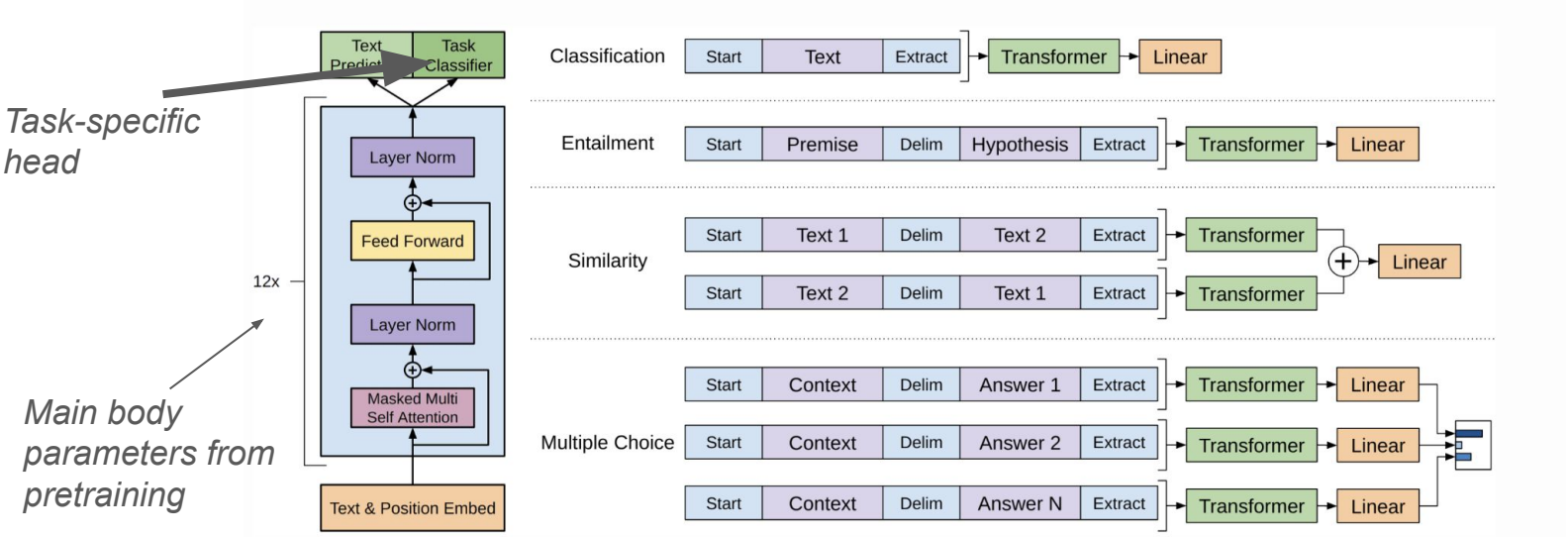
$$L = L_{xent} + \lambda \cdot L_{task}.$$



GPT Finetuning (GPT-1)

- Finetune LM parameters using a combination of two losses

$$L = L_{xent} + \lambda \cdot L_{task}.$$



BERT

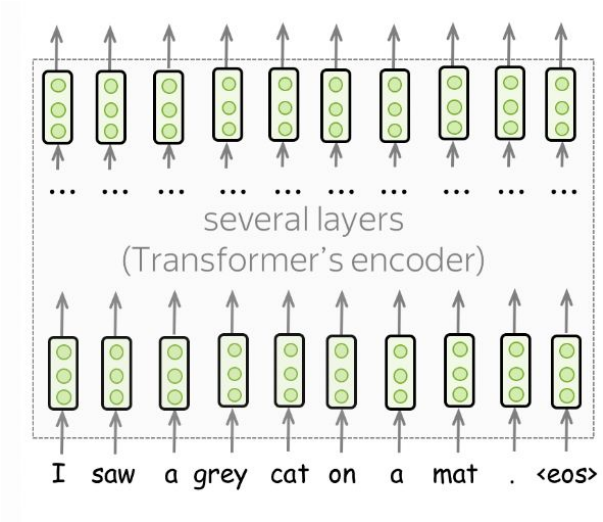
BERT

BERT (Bidirectional Encoder Pre-training for Transformers)

Key Idea: language modeling task + transformer encoder

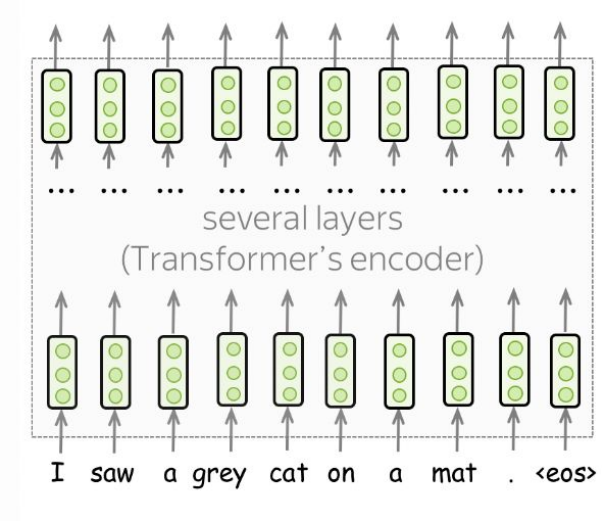
BERT

Q.: how to do LM using transformer encoder (no triangle mask) ?



BERT

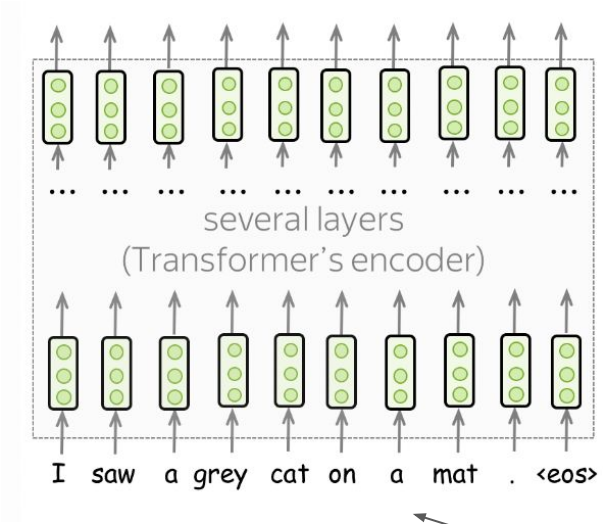
Q.: how to do LM using transformer encoder (no triangle mask) ?



- Standard l2r objective can not be used due to “lookahead”
- Let’s hide some tokens and train the model to predict masked positions

BERT

Q.: how to do LM using transformer encoder (no triangle mask) ?



- Standard l2r objective can not be used due to “lookahead”
- Let’s hide some tokens and train the model to predict masked positions

- Replace with special [MASK token]
- Classification head over final layer embeddings

BERT pretraining objective

First part: Masked Language Modeling (MLM)

Second part: model pairwise sentence dependencies

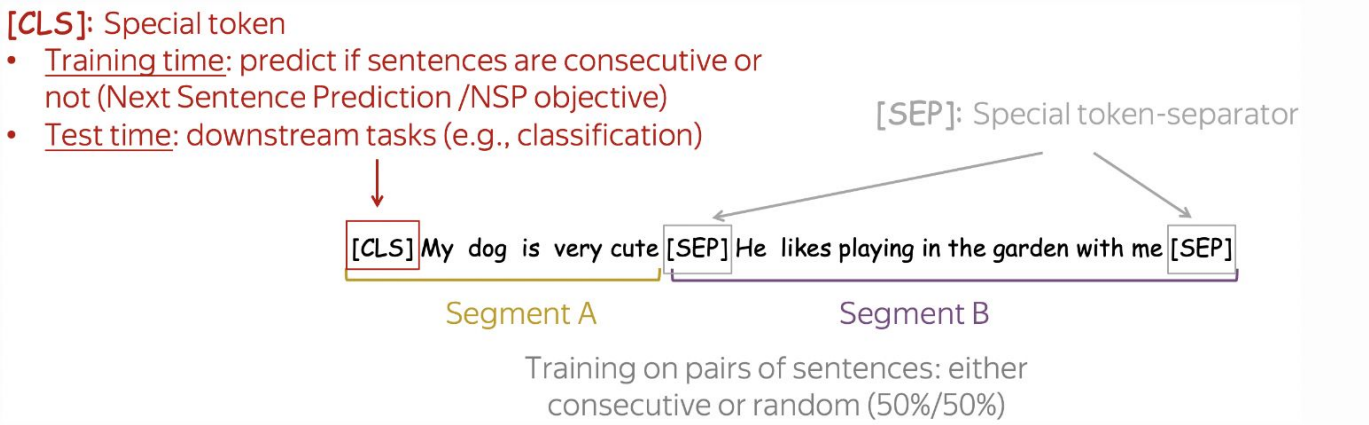
- NSP (Next Sentence Prediction)
- Whether or not two sentences directly follow each other
- [CLS] token to encode “full sentence meaning”

BERT pretraining objective

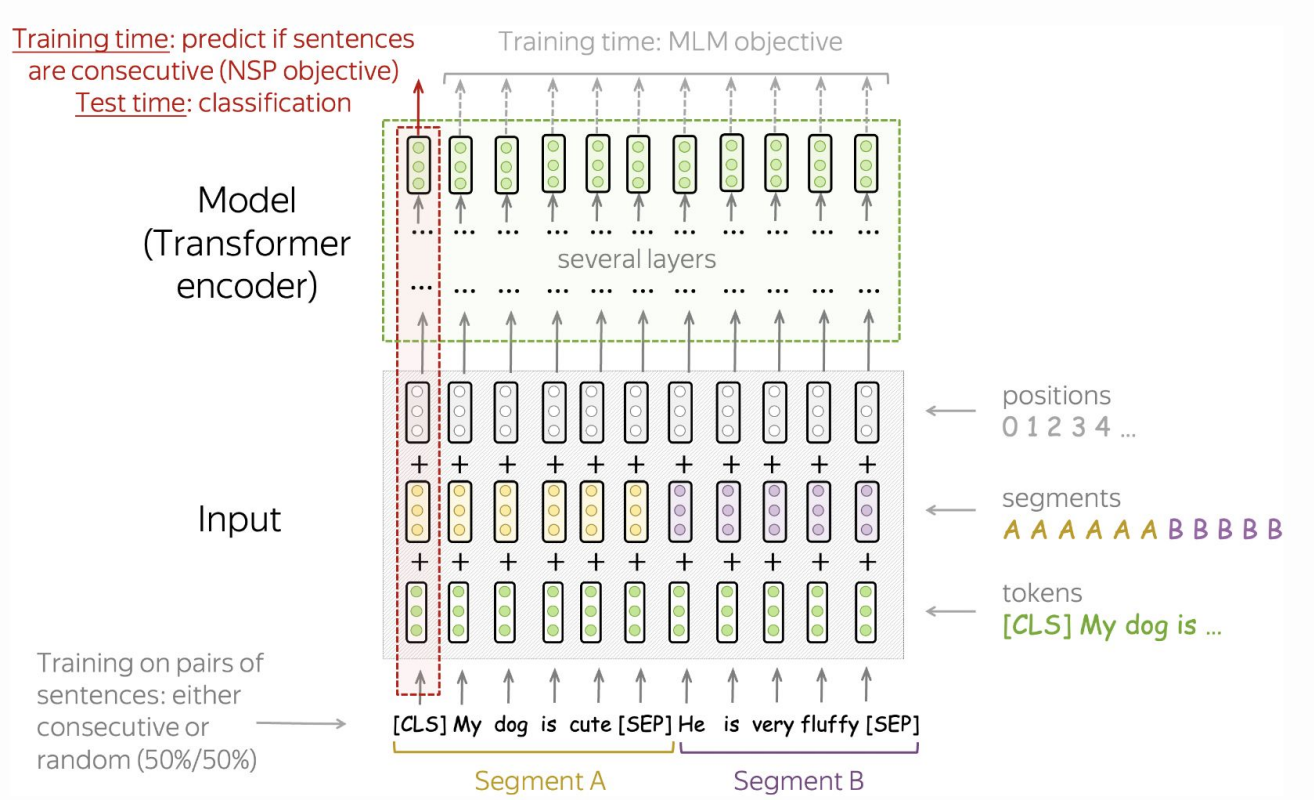
First part: Masked Language Modeling (MLM)

Second part: model pairwise sentence dependencies

- NSP (Next Sentence Prediction)
- Whether or not two sentences directly follow each other
- [CLS] token to encode “full sentence meaning”

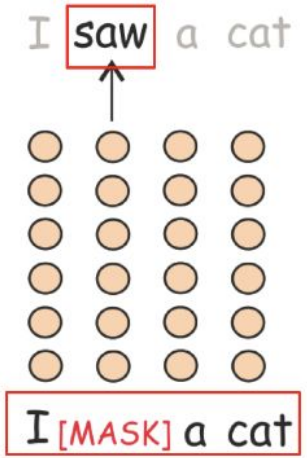


BERT Pretraining (NSP)



BERT Pretraining (MLM)

- Target: current token (the true one)
- Prediction: $P(* \mid \texttt{I [MASK] a cat})$



sees the whole text, but something is corrupted

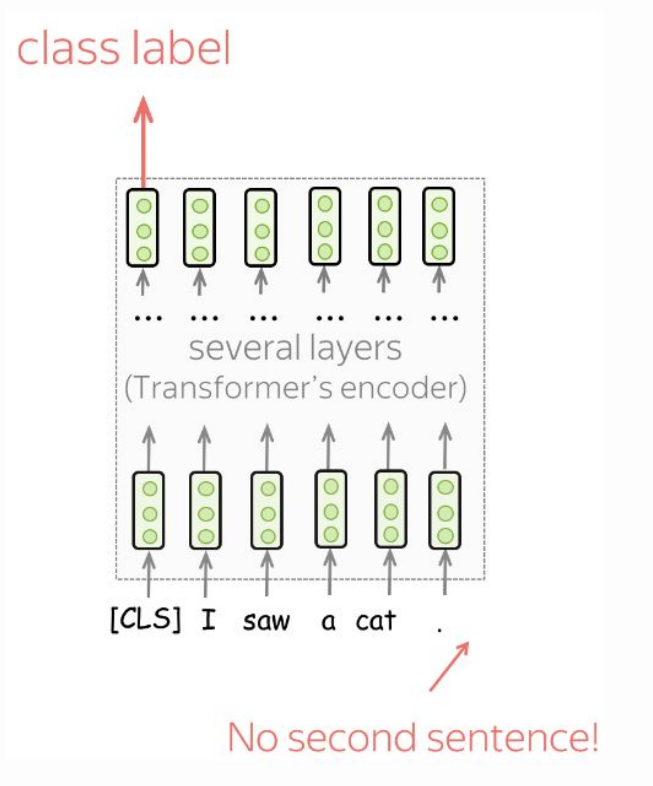
BERT Finetuning

Sentence Classification

BERT Finetuning

Sentence Classification

- Input: [CLS] + Sent
- [CLS] embedding as a feature



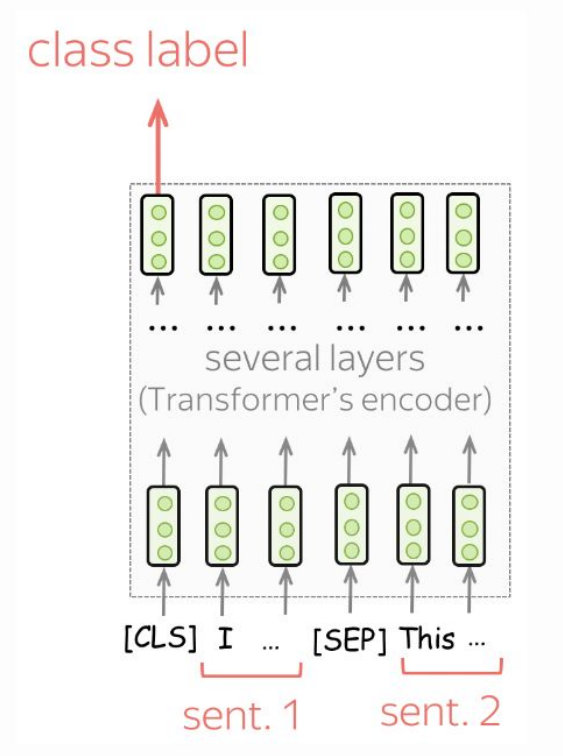
BERT Finetuning

Sentence Pair Classification

BERT Finetuning

Sentence Pair Classification

- Input: [CLS] + Sent1 + [SEP] + Sent2
- [CLS] embedding as feature



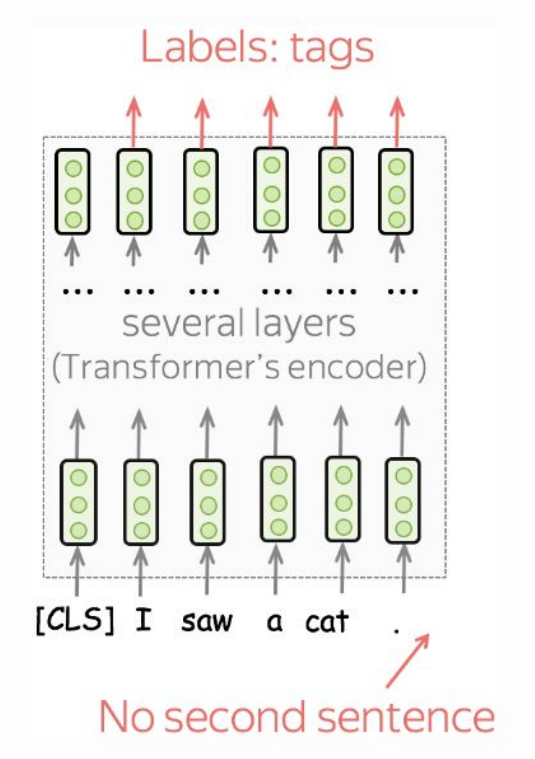
BERT Finetuning

Sentence Tagging (token classification)

BERT Finetuning

Sentence Tagging (token classification)

- Input: [CLS] + sent
- Each token's feature is its embedding



How to improve BERT

RoBERTa (BERT maximized)

RoBERTa (Robustly Optimized BERT Pretraining Approach), Liu et.al. 2019

“How to improve BERT using engineering tuning only?”

	MNLI	QNLI	QQP	RTE	SST	MRPC	CoLA	STS	WNLI	Avg
Single-task single models on dev										
$\text{BERT}_{\text{LARGE}}$	86.6/-	92.3	91.3	70.4	93.2	88.0	60.6	90.0	-	-
$\text{XLNet}_{\text{LARGE}}$	89.8/-	93.9	91.8	83.8	95.6	89.2	63.6	91.8	-	-
RoBERTa	**90.2/90.2**	**94.7**	**92.2**	**86.6**	**96.4**	**90.9**	**68.0**	**92.4**	**91.3**	-
Ensembles on test (from leaderboard as of July 25, 2019)										
ALICE	88.2/87.9	95.7	**90.7**	83.5	95.2	92.6	**68.6**	91.1	80.8	86.3
MT-DNN	87.9/87.4	96.0	89.9	86.3	96.5	92.7	68.4	91.1	89.0	87.6
XLNet	90.2/89.8	98.6	90.3	86.3	**96.8**	**93.0**	67.8	91.6	**90.4**	88.4
RoBERTa	**90.8/90.2**	**98.9**	90.2	**88.2**	96.7	92.3	67.8	**92.2**	89.0	**88.5**

RoBERTa

Baseline BERT +

- Dynamic masking
  - Generate a [MSK] segmentation at training-time
  - More diverse samples

RoBERTa

Baseline BERT +

- Dynamic masking
  - Generate a [MSK] segmentation at training-time
  - More diverse samples
- Loss function ablation
  - NSP loss function actually **degrades** performance

Model	SQuAD 1.1/2.0	MNLI-m	SST-2	RACE
Our reimplementation (with NSP loss):				
SEGMENT-PAIR	90.4/78.7	84.0	92.9	64.2
SENTENCE-PAIR	88.7/76.2	82.9	92.1	63.0
Our reimplementation (without NSP loss):				
FULL-SENTENCES	90.4/79.1	84.7	92.5	64.8
DOC-SENTENCES	90.6/79.7	84.7	92.7	65.6
$\text{BERT}_{\text{BASE}}$	88.5/76.3	84.3	92.8	64.3
$\text{XLNet}_{\text{BASE}}$ (K = 7)	–/81.3	85.8	92.7	66.1
$\text{XLNet}_{\text{BASE}}$ (K = 6)	–/81.0	85.6	93.4	66.7

RoBERTa

Baseline BERT +

- Dynamic masking
  - Generate a [MSK] segmentation at training-time
  - More diverse samples
- Loss function ablation
  - NSP loss function actually **degrades** performance
- Hyperparameter tuning
  - large batches, adjusted learning rate bring substantial benefits

bsz	steps	lr	ppl	MNLI-m	SST-2
256	1M	1e-4	3.99	84.7	92.7
2K	125K	7e-4	**3.68**	**85.2**	**92.9**
8K	31K	1e-3	3.77	84.6	92.8

Thanks for attention!

This lecture heavily uses plots and images from amazing NLP Course - For you by Lena Voita. Check it out if you wish to dig deeper!